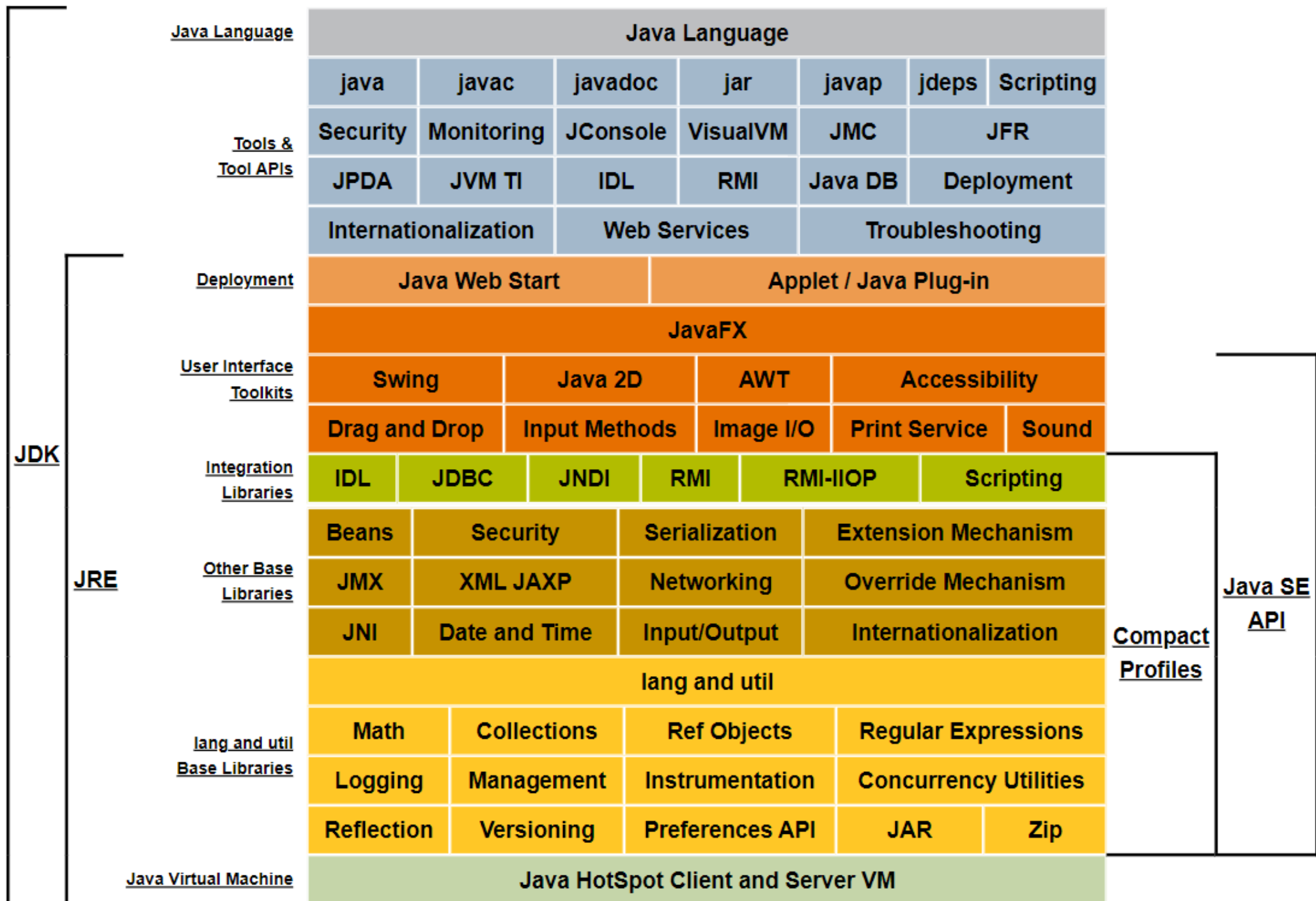


Технологии программирования

- Java Enterprise Edition -
Database Connectivity API
Persistence API



Java EE

Web Profile

Servlet

Bean
Validation

JSP

JSF



CDI beans

JTA

EJB

JPA

JAX-WS

JAX-RS

JASPIC

JSR 88

JAX-RPC

JAXB

JACC

JSR 77

JAXR

JMS

JCA

RMI

SAAJ

JAAS

JavaMail

JNDI

Архитектура Java EE приложений

Это приложения, которые разделены по функциональному принципу на изолированные модули

Обычно делятся на три уровня
(примерно так же это было по Фаулеру):

- клиентский;
- промежуточный (BLL);
- уровень доступа к данным;

Архитектура Java EE приложений

Уровень доступа к данным

Здесь можно встретить такие технологии, как:

- Java Database Connectivity API (JDBC);
- Java Persistence API;
- Java EE Connector Architecture;
- Java Transaction API (JTA).

JDBC

- The Java Database Connectivity API

Низкоуровневое API для доступа и получения данных из хранилищ данных. Типичное использование JDBC — написание SQL запросов к конкретной базе данных.

Java Persistence API

API для доступа к данным в хранилищах данных и преобразования этих данных в объекты языка программирования Java и наоборот. Гораздо более высокоуровневое API по сравнению с JDBC. Скрывает всю сложность JDBC от разработчика под капотом.

JTA

- The Java Transaction API

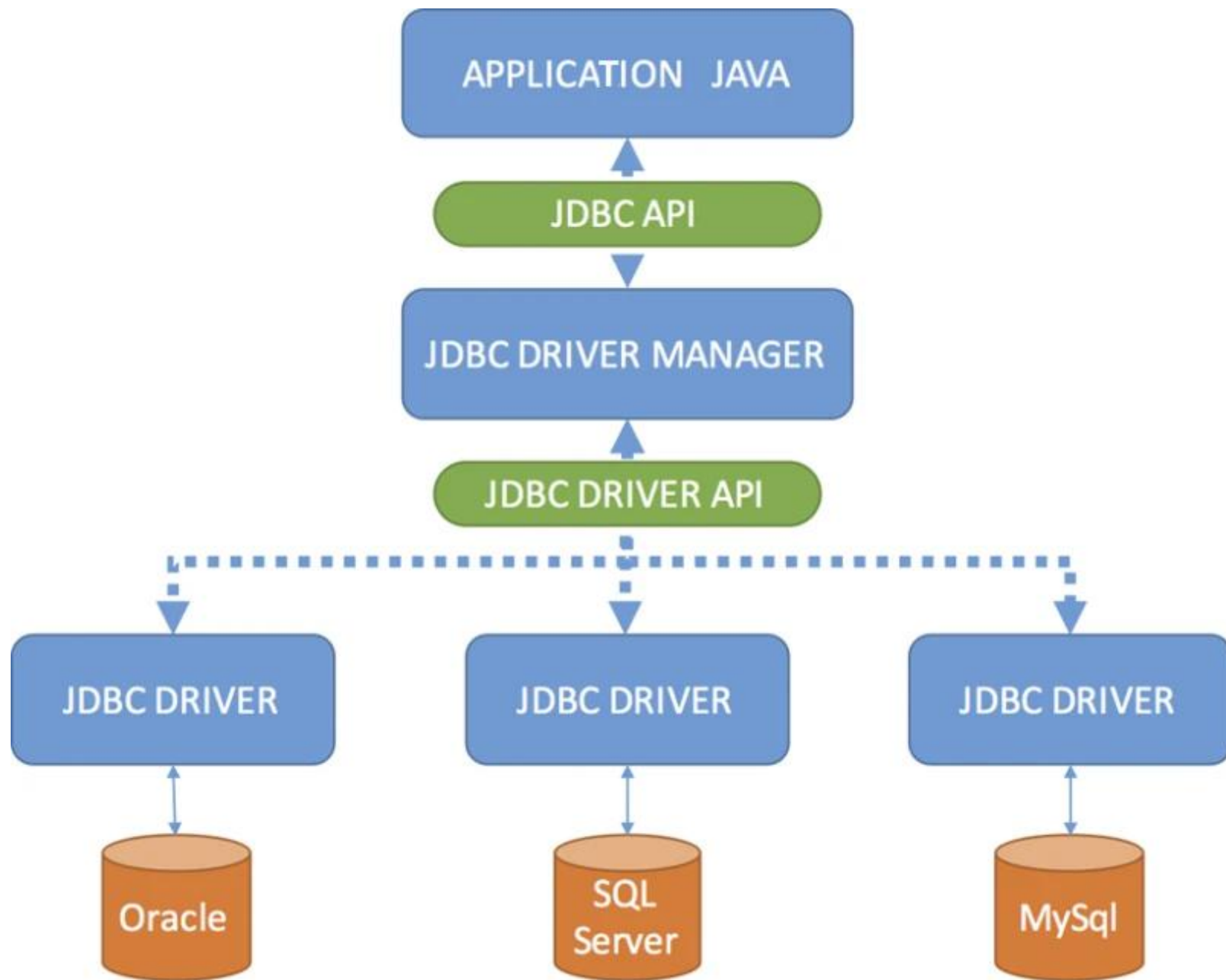
API для определения и управления транзакциями, включая распределенные транзакции, а также транзакции, затрагивающие множество хранилищ данных.

JDBC Driver Manager

Java-приложение и база данных общаются друг с другом посредством библиотеки, которая называется JDBC Driver. Это набор классов, которые реализуют JDBC API для конкретной СУБД.

Правильный JDBC драйвер же выбирается с помощью класса под названием DriverManager

Можно использовать In-Memory-DB, No-SQL-DB или даже базу данных, встроенную в Android-приложение. Java-разработчика, эти нюансы вообще не касаются. Driver Manager выберет вам правильный JDBC драйвер и все будет работать как часы.



Connection string

Для подключения протокола работы с JDBC API, необходимо использовать следующую Connection String:

jdbc:mysql://localhost:3306/db_scheme

- **mysql** – это протокол работы с сервером
- **localhost** – имя хоста в сети
- **3306** – порт, по которому идут запросы
- **db_scheme** – имя схемы (имя базы данных)

JDBC

Самая важная деталь – сам JDBC Driver для вашей базы данных. Просто так он не появится на компьютере, поэтому нужно определить зависимость чтобы система сборки её установила.

Например:

```
<dependency>
```

```
    <groupId>mysql</groupId>
```

```
    <artifactId>mysql-connector-java</artifactId>
```

```
    <version>8.0.29</version>
```

```
</dependency>
```

Доступ к базе данных

После того как зависимости подключены, необходимо создать подключение к базе.

```
1 Connection connection = DriverManager.getConnection(  
2     "jdbc:mysql://localhost:3306/test",  
3     "login", "password");
```

Логин и пароль, разумеется, подставляется к вашей локальной базе данных.

```
1 package org.example;
2
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5 import java.sql.ResultSet;
6 import java.sql.Statement;
7
8 public class JdbcApplicatin {
9     public static void main(String[] args) throws Exception{
10         Connection connection = DriverManager.getConnection(
11             "jdbc:mysql://localhost:3306/test",
12             "root", "secret");
13
14         Statement statement = connection.createStatement();
15         ResultSet results = statement.executeQuery("SELECT * FROM user");
16
17         while (results.next()) {
18             Integer id = results.getInt(1);
19             String name = results.getString(2);
20             System.out.println(results.getRow() + ". " + id + "\t" + name);
21         }
22         connection.close();
23     }
24 }
```

Statements

Все SQL-запросы можно условно разделить на две группы:

- Получение данных — к ним относится оператор SELECT.
- Изменение данных — к ним относятся операторы INSERT, UPDATE и DELETE.

Statements

- Для первой группы используется уже знакомый нам метод интерфейса **Statement** — executeQuery(). В принципе для начала этого метода вполне достаточно. Он покрывает очень большой процент запросов, которые вам придется использовать на реальной работе.

Statements

- Для второй группы запросов нужно использовать другой метод интерфейса **Statement** — executeUpdate(). В отличие от метода executeQuery(), который возвращает **ResultSet**, этот метод возвращает целое число, которое говорит сколько строк в таблице было изменено при исполнении вашего запроса.

Callable Statements

```
1. CallableStatement callableStatement =  
2.     connection.prepareCall("{call calculateStatistics(?, ?)}",  
3.         ResultSet.TYPE_FORWARD_ONLY,  
4.         ResultSet.CONCUR_READ_ONLY,  
5.         ResultSet.CLOSE_CURSORS_OVER_COMMIT  
6.     );
```

- CallableStatement используется для вызова хранимых процедур в базе данных.
- Хранимая процедура похожа на функцию или метод в классе, за исключением того, что она находится в базе данных. Некоторые тяжелые операции с базой данных могут выиграть в производительности от выполнения в том же пространстве памяти, что и сервер базы данных, в качестве хранимой процедуры.

DataSource и ConnectionPoolDataSource

- Оба интерфейса входят в пакет `javax.sql` и должны быть реализованы поставщиками JDBC классов (драйверов). Основное назначение интерфейсов `DataSource` и `ConnectionPoolDataSource` состоит в предоставлении возможности получения соединения с базой данных абстрагируясь от местоположения сервера СУБД и типа драйвера конкретного производителя. Интерфейсы определяют ряд обязательных для реализации методов, в том числе и метод `getConnection()`. Объекты, получаемые от реализации данных интерфейсов используются для задания параметров соединения с базой данных и установки соединения в виде объекта типа `Connection`.

DataSource и ConnectionPoolDataSource

- JDBC DataSource объекты используются для получения физического соединения с базой данных и являются альтернативой DriverManager. При этом нет необходимости регистрировать драйвер. Необходимо только установить соответствующие параметры для установки соединения и выполнить метод getConnection(). При создании объекта типа DataSource локально параметры соединения задаются соответствующими методами, предусмотренными поставщиком драйвера. Эти методы не определены интерфейсом DataSource, т.к. параметры для соединения с СУБД разных производителей могут отличаться как по типу, так и по количеству.

DataSource и ConnectionPoolDataSource

```
import java.sql.*;
import javax.sql.*;
import oracle.jdbc.driver.*;
import oracle.jdbc.pool.*;

public class DataSource {

    public static void main(String[] args) {
        try {
            OracleDataSource ods = new OracleDataSource();

            ods.setUser("stud");
            ods.setPassword("stud");
            ods.setDriverType("thin");
            ods.setDatabaseName("orbis");
            ods.setServerName("localhost");
            ods.setPortNumber(1521);

            Connection conn = ods.getConnection();
            System.out.println("Connection successful!!!");

        } catch (SQLException se) {
            se.printStackTrace();
        }
        System.out.println("Goodbye!");
    }
}
```

JPA

- JPA (Java Persistence API) это спецификация Java EE и Java SE, описывающая систему управления сохранением java объектов в таблицы реляционных баз данных в удобном виде. Сама Java не содержит реализации JPA, однако есть существует много реализаций данной спецификации от разных компаний (открытых и нет). Это не единственный способ сохранения java объектов в базы данных (ORM систем), но один из самых популярных в Java мире.

JDO

- JDO более общая спецификация которая описывает ORM для любых возможных баз и хранилищ. В принципе можно рассматривать JPA как специализированную на релятивистских баз часть спецификации JDO, даже при том что API этих двух спецификаций не полностью совпадает. Также отличаются «разработчики» спецификаций — если JPA разрабатывается как JSR, то JDO сначала разрабатывался как JSR, теперь разрабатывается как проект Apache JDO.

Hibernate

- Hibernate одна из самых популярных открытых реализаций последней версии спецификации (JPA 2.1). Даже скорее самая популярная, почти стандарт де-факто. То есть JPA только описывает правила и API, а Hibernate реализует эти описания, впрочем у Hibernate (как и у многих других реализаций JPA) есть дополнительные возможности, не описанные в JPA (и не переносимые на другие реализации JPA).

JPA Entity

- Entity это легковесный хранимый объект бизнес логики (persistent domain object). Основная программная сущность это entity класс, который так же может использовать дополнительные классы, который могут использоваться как вспомогательные классы или для сохранения состояния entity.

JPA Entity

```
@Entity
public class User {
    private static Log log = LogFactory.getLog(User.class);

    @Id
    @GeneratedValue
    private int id;

    private String userName;
    private String firstName;
    private String lastName;
    @Transient
    private String fullName;

    // Standard getters/setters
}
```

JPA EntityManager

- EntityManager это интерфейс, который описывает API для всех основных операций над Entity, получение данных и других сущностей JPA. По сути главный API для работы с JPA. Основные операции:
 - 1) Для операций над Entity: persist, merge, remove, refresh), detach, lock
 - 2) Получение данных: find, createQuery, createNamedQuery, createNativeQuery, contains, createNamedStoredProcedureQuery, createStoredProcedureQuery
 - 3) Получение других сущностей JPA: getTransaction, getEntityManagerFactory, getCriteriaBuilder, getMetamodel, getDelegate
 - 4) Работа с EntityGraph: createEntityGraph, getEntityGraph
 - 4) Общие операции над EntityManager или всеми Entities: close, isOpen, getProperties, setProperty, clear

JPA EntityManager

```
package es.art83.persistence.jpa.inheritance;

import javax.persistence.EntityManager;
import javax.persistence.Persistence;

public class InheritanceMain {

    public static void main(String[] args) {
        EntityManager entityManager = Persistence.createEntityManagerFactory("BBDD").createEntityManager();
        entityManager.getTransaction().begin();
        entityManager.persist(new C1A(1,1,1,1));
        entityManager.persist(new C1A(2,2,2,2));
        entityManager.persist(new C1B(3,3,3,3));
        entityManager.persist(new C1B(4,4,4,4));
        entityManager.getTransaction().commit();
    }
}
```

JPA Entity life-cycle

У Entity объекта существует четыре статуса жизненного цикла: new, managed, detached, или removed. Их описание:

- 1) new — объект создан, но при этом ещё не имеет сгенерированных первичных ключей и пока ещё не сохранен в базе данных,
- 2) managed — объект создан, управляется JPA, имеет сгенерированные первичные ключи,
- 3) detached — объект был создан, но не управляется (или больше не управляется) JPA,
- 4) removed — объект создан, управляется JPA, но будет удален после commit'а транзакции.